

CS3334 - Data Structures

Lab 3

Outline

- Queue and Hash Exercises
- Task Manager
- Printer Queue

Exercise 1

Determine whether each of the following characteristics apply to **a stack, a queue, both, or none**.

- a. An element is inserted at a special place called the top.
- b. An element is inserted at a special place called the rear.
- c. The structure can hold only one type of data element.
- d. An element is deleted at the front.
- e. The i -th position may be deleted.
- f. An element is deleted at the top.
- g. The structure is a LIFO structure.
- h. The structure is a FIFO structure.

Exercise 2

For a queue implemented using a fixed-size array of capacity K with front and rear indices (enabling circular), which condition correctly identifies that the circular queue is full?

- a. $\text{rear} == \text{front}.$
- b. $\text{front} == (\text{rear} + 1).$
- c. $(\text{rear} + 1) \% K == \text{front}.$
- d. $\text{rear} == K - 1.$

Exercise 3

In a standard array-based linear queue of capacity N , suppose we perform k consecutive enqueue operations followed by j consecutive dequeue operations (where $k > j > 0$ and $k < N$). Where do the front and rear pointers currently point relative to the array?

- a. front points to index 0, and rear points to index $k-1$.
- b. front points to index j , and rear points to index $k-1$.
- c. front points to index $j-1$, and rear points to index k .
- d. front points to index k , and rear points to index j .

Exercise 4

What is the time complexity of both standard enqueue and dequeue operations in a properly implemented queue using a fixed-size (N) array?

Exercise 5

Given an integer ***k*** and a queue of integers, the task is to reverse the order of the first ***k*** elements of the queue, leaving the other elements in the same relative order.

Only following standard operations are allowed on queue.

- enqueue(*x*) : Add an item *x* to rear of queue
- dequeue() : Remove an item from front of queue
- size() : Returns number of elements in queue.
- front() : Finds front item but not remove it.

Example:

Input: *k* = 5, queue = {10, 20, 30, 40, 50, 60, 70, 80, 90, 100} (10 is the front)

Output: {50, 40, 30, 20, 10, 60, 70, 80, 90, 100}

```
void reverseFirstK(int k, queue<int>& Queue) { . . . }
```

Exercise 6

According to lecture, a hash table consists of several components.
Which of these was **NOT** one of those components?

1. An array
2. Collision handling
3. Encryption
4. A hash function

Exercise 7

Given the following input

(4322, 1334, 1471, 9679, 1989, 6171, 6173, 4199)

and the hash function **$x \bmod 10$** ,

which of the following statement(s) are true?

1. 9679, 1989, 4199 hash to the same value
2. 1471, 6171 hash to the same value
3. All elements hash to the same value
4. Each element hashes to a different value

Exercise 8

The keys 12, 18, 13, 2, 3, 23, 5 and 15 are inserted into an initially empty hash table of length 10 using open addressing with hash function $h(k) = k \bmod 10$ and **linear probing**.

What is the resultant hash table?

0	
1	
2	
3	
4	
5	
6	
7	
8	
9	

Exercise 9

A hash table of length 10 uses open addressing with hash function $h(k)=k \bmod 10$, and linear probing. After inserting 6 values into an empty hash table, the table is as shown below.

Which one of the following choices gives a possible order in which the key values could have been inserted in the table?

(A) 46, 42, 34, 52, 23, 33

(B) 34, 42, 23, 52, 33, 46

(C) 46, 34, 42, 23, 52, 33

(D) 42, 46, 33, 23, 34, 52

0	
1	
2	42
3	23
4	34
5	52
6	46
7	33
8	
9	

Task Manager

Description

One day, John was curious about his laptop and wondered how the computer handles many different tasks at the same time. After reading some books, he figured out that the CPU follows first-come-first-serve policy and uses queue to handle different tasks.

In this model, each task has an arriving time **R**, a required processing time **W**, and priority level **P** (high priority or low priority).

CPU will maintain two queues, one queue contains tasks of high priority and the other contains tasks of low priority.

When a new task arrives, the CPU will put it into the queue according to the priority. And when the CPU finishes the current task, it will execute the next task extracted from the high priority queue and from the low priority queue if the high priority queue is empty.

Can you write a program to compute the finishing time of each task?

Task Manager

Input

The input contains multiple test cases. For each test case, the first line contains an integer n , indicating that there are n tasks.

The following n ($1 \leq n \leq 100$) lines give the task information, where the i -th line contains three integers R_i , W_i , P_i , indicating the information of the i -th task, where $0 \leq R_i \leq 1000$, $1 \leq W_i \leq 100$, $0 \leq P_i \leq 1$ and $P_i=0$ means high priority.

It is guaranteed that the sequence of arrival times is non-decreasing (if two tasks have the same arrival time and same priority, the task who is earlier in the input is considered to have arrived earlier).

Output

For each test case, print a single line containing the finishing time of each task (Separate the integers by a space and no more space after the last integer in the line).

Task Manager

Sample Input	Sample Output
3 0 4 1 0 2 0 3 4 0 3 0 4 0 0 2 0 3 4 0 5 3 4 1 5 3 0 5 7 1 6 3 0 6 6 0 5 3 4 1 5 3 0 5 7 1 14 3 0 26 6 0	6 2 10 4 6 10 7 10 26 13 19 7 10 17 20 32

Printer Queue

Description

The only printer in the computer science students' union is experiencing an extremely heavy workload. Sometimes there are a hundred jobs in the printer queue, and you may have to wait for hours to get a single page of output.

Because some jobs are more important than others, the Hacker General has invented and implemented a simple priority system for the print job queue. Now, each job is assigned a priority between 1 and 9 (with 9 being the highest priority, and 1 being the lowest), and the printer operates as follows.

- The first job J in queue is taken from the queue.
- If there is some job in the queue with a higher priority than job J, then move J to the end of the queue without printing it.
- Otherwise, print job J (and do not put it back in the queue).

In this way, all those important muffin recipes that the Hacker General is printing get printed very quickly. Of course, those annoying term papers that others are printing may have to wait for quite some time to get printed, but that's life.

Your problem with the new policy is that it has become quite tricky **to determine when your print job will actually be completed**. You decide to write a program to figure this out.

The program will be given the current queue (as a list of priorities) as well as the position of your job in the queue, and must then calculate how long it will take until your job is printed, assuming that no additional jobs will be added to the queue. To simplify matters, we assume that printing a job always takes exactly one minute, and that adding and removing jobs from the queue is instantaneous.

Printer Queue

Input

One line with a positive integer: the number of test cases (at most 100).

Then for each test case:

- One line with two integers n and m , where n is the number of jobs in the queue ($1 \leq n \leq 100$) and m is the position of your job ($0 \leq m \leq n - 1$). The first position in the queue is number 0, the second is number 1, and so on.
- One line with n integers in the range 1 to 9, giving the priorities of the jobs in the queue. The first integer gives the priority of the first job, the second integer the priority of the second job, and so on.

Output

For each test case, print one line with a single integer; the number of minutes until your job is completely printed, assuming that no additional print jobs will arrive.

Printer Queue

Sample Input	Sample Output
3 1 0 5 4 2 1 2 3 4 6 0 1 1 9 1 1 1	1 2 5